O que é (e o que não é) um microsserviço

Arquitetura de Software SAAS (8512) · EC7 · SETREM 2026-2 · Encontro 11, 15/10/2026 · síntese do encontro

> **O que fazer com esta síntese**
>
> - Ler depois do commit `observacoes`, antes do fechamento das 21h15 (15 min).
> - Trazer para o fechamento uma resposta para a coluna **"Onde o CampoConta não cumpre?"** da tabela da seção 2.
> - Até 17/10, 23h59: o `docs/servicos-rascunho.md` do seu PI completo, como pede o fim de `experimentos.md` (as caixas e as cinco perguntas; a 5ª é sobre esta síntese).

Por que isso interessa ao seu projeto: na defesa da Etapa 2 (29/10), "por que isso são dois serviços, e não um?" é pergunta certa. Quem responde só "porque são microsserviços" não respondeu.

1. A definição

> Um **microsserviço** é um processo que cuida de **um assunto do negócio**, tem **os próprios dados**, pode ser **implantado sem os outros** e conversa com os outros **só pela rede** (FOWLER; LEWIS, 2014; NEWMAN, 2021).

Uma **arquitetura de microsserviços** é um produto feito de vários desses, combinados. Você acabou de operar dois: `producao` e `cobranca`. O `gateway` é a porta de entrada que os combina.

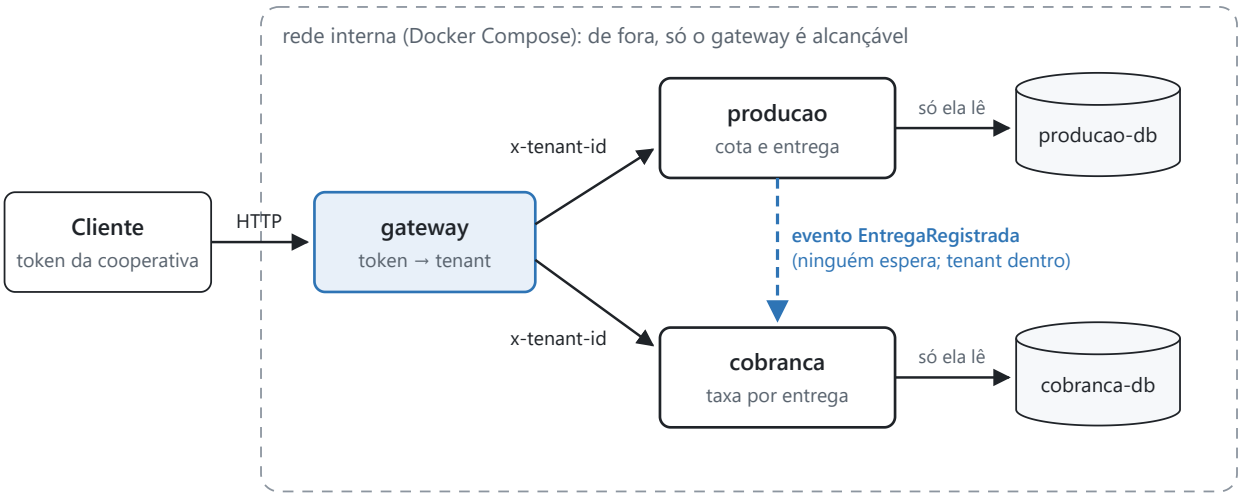


Figura 1. O CampoConta que você operou hoje. A seta tracejada é a única conversa entre os dois serviços de domínio, e é assíncrona: a produção responde ao balanceiro sem esperar a cobrança.

2. As cinco características, e onde você as viu hoje

Característica	O que quer dizer	Onde você viu hoje	Onde o CampoConta não cumpre?
Em torno de um assunto do negócio	O corte segue o domínio (cota e entrega; taxa), não a técnica (tela, regra, banco). É o bounded context do Encontro 08 virando processo	Exp. 1: a entrega passa pela produção e a taxa nasce na cobrança	
Dono dos próprios dados	Ninguém lê o banco do outro; o que um sabe do outro chega por chamada ou por evento	Exp. 3: o `JOIN` falha, e o relatório é montado no gateway	
Implantação independente	Dá para mudar, reiniciar ou trocar um serviço sem parar os outros	Exp. 4: a cobrança para e a entrega continua	
Conversa só pela rede	Síncrona (pedido e resposta, alguém espera) ou assíncrona (mensagem numa fila, ninguém espera)	Exp. 1 e 4: o gateway chama por HTTP; a produção avisa a cobrança por evento	
Projetado para falhar	Todo outro serviço pode estar lento ou fora; cada chamada tem prazo e plano B	Exp. 4: relatório parcial em 2 s; Exp. 5: evento repetido descartado	

> **A última coluna está vazia de propósito.** Procure onde uma característica é só parcialmente verdade. Por onde começar: no `compose.yaml`, compare as linhas `image:` e `volumes:` dos três serviços; em `producao/server.js`, procure onde o relay descobre o endereço da cobrança; e releia a seção 5. Uma linha com o arquivo onde você viu já basta.

3. O que um microsserviço não é

- **Não é "serviço pequeno".** O tamanho não define nada; o assunto único e a independência, sim.
- **Não é a única opção decente.** Um **monólito modular** (um processo só, com módulos que não leem as tabelas uns dos outros) tem a maior parte dos ganhos de organização sem o custo de operar vários processos. Para muito produto, é a escolha certa.
- **O pior dos dois mundos tem nome: monólito distribuído.** São serviços separados que só funcionam juntos: um não sobe sem o outro, toda mudança exige implantar todos. Paga-se o custo da rede sem ganhar a independência.

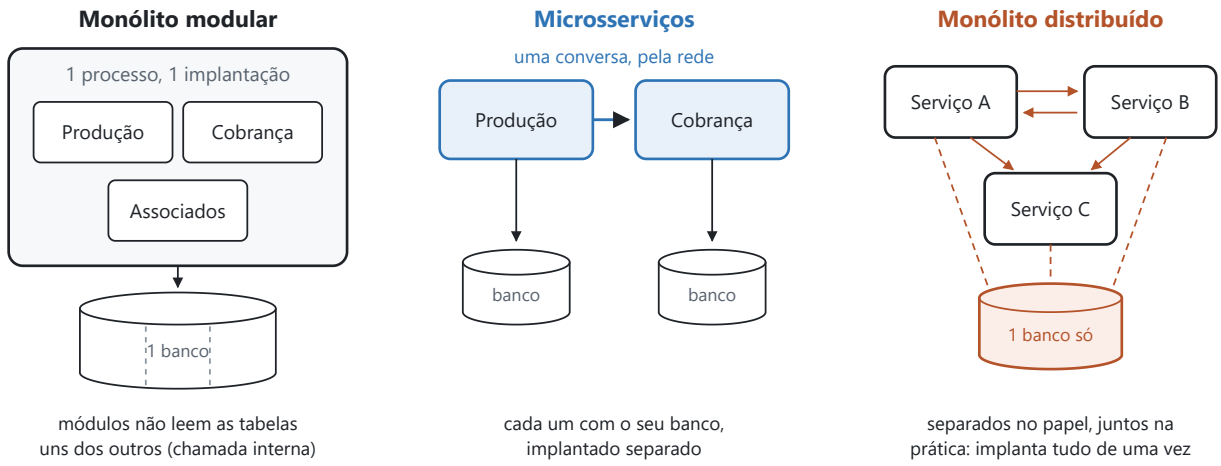


Figura 2. A diferença está nas setas e nos bancos, não no número de caixas. No monólito distribuído, três serviços dividem um banco e se chamam o tempo todo: uma mudança em um obriga a implantar os três.

4. Microsserviço num SaaS multi-tenant

Num produto multi-tenant, o tenant precisa **atravessar** os serviços sem se perder.

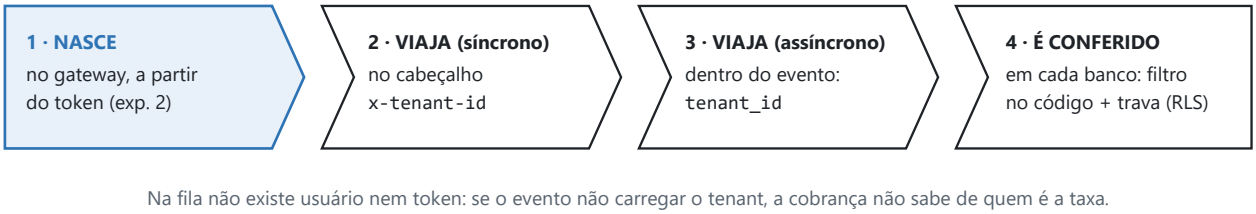


Figura 3. Se um desses quatro passos falhar em um serviço, o isolamento do produto inteiro falha ali.

5. Mensageria: fila, evento e broker

Quando ninguém precisa da resposta agora, a comunicação pode ser **assíncrona**: um serviço deixa a mensagem e o outro processa no seu ritmo. Para aprofundar: LISBOA (2021), na Biblioteca Virtual.

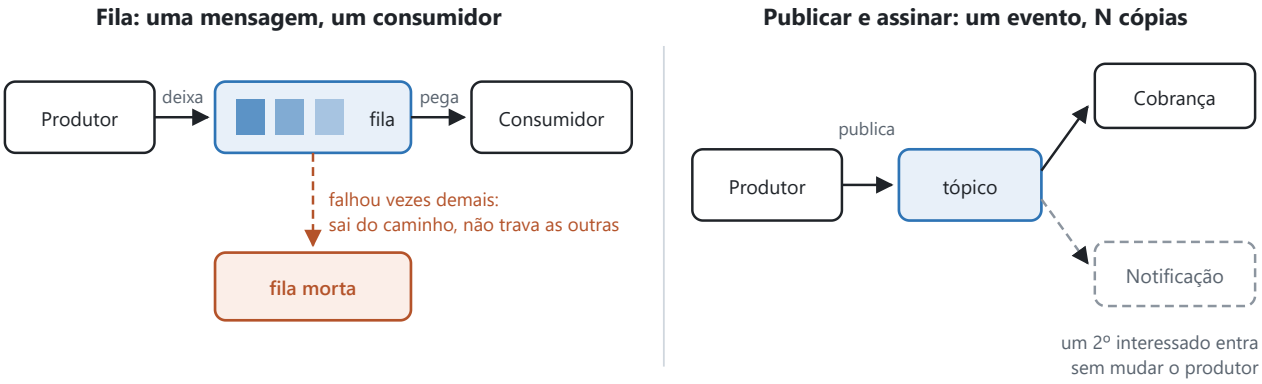


Figura 4. Na fila, cada mensagem é processada por um consumidor só. Em publicar e assinar, cada interessado recebe uma cópia do evento. O broker é o programa que guarda e entrega as mensagens, com reentrega e fila morta.

- **"Pelo menos uma vez":** quase toda mensageria garante que a mensagem chega, não que chega uma vez só. Por isso o consumidor precisa ser **idempotente**: receber a mesma mensagem duas vezes tem de ter o mesmo efeito que receber uma (exp. 5).

- **O CampoConta usa uma tabela como fila**, não um broker: a produção grava o evento na tabela `outbox` na mesma transação da entrega, e um laço chamado relay lê essa tabela e entrega cada evento à cobrança. Com um produtor e um consumidor, a tabela basta e cabe na mesma transação. Ela deixa de bastar quando aparece o segundo interessado no evento ou quando uma mensagem ruim trava as outras, porque não há fila morta.

> **De quem é cada parte.** Escolher entre tabela, fila e publicar/assinar é **decisão de arquitetura, desta disciplina**: se a comunicação do seu PI for assíncrona, o ADR de comunicação da Etapa 2 diz o mecanismo e as garantias (como o consumidor evita o efeito em dobro; o que acontece com a mensagem que falha sempre). Provisionar e operar um broker gerenciado é o que você aprofunda em **Cloud Computing**.

6. Quando não usar

Separar em serviços cobra um preço fixo: mais processos para subir, mais bancos para cuidar, rede no meio de cada conversa, consistência eventual (o dado fica certo em todos os lugares, mas não no mesmo instante, como a taxa que aparece depois da entrega), testes mais difíceis. O caso SafraLog (Encontro 09) chamou isso de "plantão novo".

> **Prefira começar por um monólito modular quando:**
> - a equipe é pequena (no PI: 1 a 4 pessoas);
> - o domínio ainda está mudando e você não sabe onde ficam as fronteiras;
> - nenhuma parte precisa escalar, falhar ou ser implantada separada das outras.
>
> Separe quando uma dessas três coisas deixar de ser verdade, e registre o porquê num ADR.

Referências

FOWLER, Martin; LEWIS, James. **Microservices guide**. 2014. Disponível em: <https://martinfowler.com/microservices/>. Acesso em: 16 jul. 2026.

LISBOA, Flavio. **Arquitetura de software distribuído: boas práticas para um mundo de microsserviços**. São Paulo, SP: Casa do Código, 2021. E-book. Disponível em: <https://plataforma.bvirtual.com.br>. Acesso em: 11 out. 2026.

NEWMAN, Sam. **Building microservices: designing fine-grained systems**. 2. ed. Sebastopol: O'Reilly, 2021.

O CampoConta é material de aula, não base de PI. A versão em texto desta síntese está em `aula/enc11/o-que-e-microsservico.md`, no mesmo repositório.